

- -Fp: This performs a backup on the replica, maintaining the same data structure present on the primary.
 - -Xs: This opens a second connection to the primary server and starts the transfer of the WAL segments at the same time as the backup is performed.
 - -P: This shows the progress of the backup.
 - -S: This is the slotname created on the primary server.
 - -R: This creates the `standby.signal` file and adds the connection settings to the `PostgreSQL.auto.conf` file:
 - `postgres@pg2:~/16/main$ cat postgresql.auto.conf`
 - `# Do not edit this file manually!`
 - `# It will be overwritten by the ALTER SYSTEM command.`
 - `primary_conninfo = 'user=replicarole password=SuperSecret channel_binding=disable host=192.168.122.10 port=5432 sslmode=disable sslcompression=0 sslcertmode=disable sslsni=1 ssl_min_protocol_version=TLSv1.2 gssencmode=disable krbsrvname=postgres target_session_attrs=any load_balance_hosts=disable'`
 - `primary_slot_name = 'master'`
3. Now let's start the PostgreSQL service on the replica machine, and physical replication should work. As the root user, let's execute the following:

```
root@pg2:/var/lib/postgresql/16# systemctl start postgresql
```

As we can see from the PostgreSQL log file (`/var/log/postgresql/postgresql-16-main.log`), the replica machine started in standby mode and read-only mode:

```
2023-05-22 13:27:29.823 UTC [1244] LOG:  entering standby mode
2023-05-22 13:27:29.832 UTC [1244] LOG:  redo starts at 0/2000028
2023-05-22 13:27:29.835 UTC [1244] LOG:  consistent recovery state
reached at 0/2000100
2023-05-22 13:27:29.835 UTC [1241] LOG:  database system is ready to
accept read-only connections
```

4. Let's connect to the replica server and try to see whether everything has been replicated:

```
postgres=# \l
```

```

List of
databases
```

```

      Name      | Owner  | Encoding | Locale Provider | Collate |
Ctype         | ICU Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+
postgres      | postgres | UTF8      | libc            | en_US.UTF-8 |
en_US.UTF-8   |          |           |                 |             |
template0     | postgres | UTF8      | libc            | en_US.UTF-8 |
en_US.UTF-8   |          |           | =c/postgres     | +           |
              |          |           |                 |             |
              |          | postgres=CtC/postgres
template1     | postgres | UTF8      | libc            | en_US.UTF-8 |
en_US.UTF-8   |          |           | =c/postgres     | +           |
              |          |           |                 |             |
              |          | postgres=CtC/postgres
(3 rows)

```

5. Let's try to create a table:

```

postgres=# create table test_table (id integer);
ERROR:  cannot execute CREATE TABLE in a read-only transaction

```

As we can see, the server is now in read-only mode.

Replica monitoring

After successfully installing our first asynchronous replica server, let's look at how we can monitor the health of our replica. PostgreSQL offers us a view through which we can monitor the status of replicas in real time; its name is `pg_stat_replication`. This view must be queried by connecting to the primary node.

For example, if we connect to the main node, we can see the following:

```

postgres=# \x
Expanded display is on.
postgres=# select * from pg_stat_replication ;
-[ RECORD 1 ]-----+-----
pid           | 1720
usesysid      | 16388
username      | replicarole
application_name | walreceiver

```

```

client_addr      | 192.168.122.11
client_hostname  |
client_port      | 41690
backend_start    | 2023-05-22 13:27:29.849+00
backend_xmin     |
state            | streaming
sent_lsn         | 0/3000148
write_lsn        | 0/3000148
flush_lsn        | 0/3000148
replay_lsn       | 0/3000148
write_lag        |
flush_lag        |
replay_lag       |
sync_priority    | 0
sync_state       | async
reply_time       | 2023-05-22 13:30:59.928374+00

```

Using this view, we have a lot of information that we need in order to know whether our `stand_by` server is in excellent health.

For example, we can see that the last reply message received from the replica server is `2023-05-22 13:30:59.928374+00`, and we can see, thanks to the difference between the `sent_lsn` value and the `replay_lsn` value, that our replication server is perfectly aligned. For further information about `pg_stat_replication`, please refer to the official documentation (<https://www.postgresql.org/docs/current/monitoring-stats.html#PG-STAT-REPLICATION-VIEW>).

Synchronous replication

So far, we have talked about asynchronous replication; this means that the primary server passes information to the replica standby without being sure that the standby server has replicated the data. In asynchronous replication, the primary server does not wait for the replica server to actually replicate the data. In synchronous replication, when the primary performs a commit, all the replicated servers synchronously commit. In synchronous replication, after the execution of the commit, we are sure that the data is replicated on the primary and all the replicas. When we want to achieve synchronous replication, it is good practice to have all identical machines and a good network connection between the machines; otherwise, performance can become slow.

PostgreSQL settings

Starting with what has been done for asynchronous replication and simply changing some settings, it is possible to change from asynchronous replication to synchronous replication.

Primary server

On the primary server, we have to check whether the `synchronous_commit` parameter is set to on. Now, `synchronous_commit = on` is the default value on a new PostgreSQL installation.

After setting this parameter, we must add the `synchronous_standby_names` parameter, listing the names of all standby servers that will replicate the data synchronously. We can also use the `'*'` wildcard, thus indicating to PostgreSQL that each standby server can potentially have a synchronous replica. For example, to transform the primary of the previous example so that it can support asynchronous replication for the `pg2` server, we have to write this:

```
synchronous_standby_names = 'pg2'
synchronous_commit = on
```

After this, we need to restart our server:

```
# systemctl restart postgresql
```

Standby server

On the standby server, we have to add a parameter to the connection string to the primary so that the primary knows from whom the reply request comes. We need to edit the `postgresql.auto.conf` file; it is currently as follows:

```
# Do not edit this file manually!
# It will be overwritten by the ALTER SYSTEM command.
primary_conninfo = 'user=replicarole password=SuperSecret channel_
binding=disable host=192.168.122.10 port=5432 sslmode=disable
sslcompression=0 sslcertmode=disable sslsni=1 ssl_min_protocol_
version=TLSv1.2 gssencmode=disable krbsrvname=postgres target_session_
attrs=any load_balance_hosts=disable'
primary_slot_name = 'master'
```

We need to change this to the following:

```
# Do not edit this file manually!
# It will be overwritten by the ALTER SYSTEM command.
```

```
primary_conninfo = 'user=replicarole password=SuperSecret channel_
binding=disable host=192.168.122.10 port=5432 sslmode=disable
sslcompression=0 sslcertmode=disable sslsni=1 ssl_min_protocol_
version=TLSv1.2 gssencmode=disable krbsrvname=postgres target_session_
attrs=any load_balance_hosts=disable application_name=pg2'
primary_slot_name = 'master'
```

We have added the `application_name=pg2` option.

After doing this, let's restart the standby server. Now if we get back on the primary server and recheck the `pg_stat_replication` view, we will see this result:

```
postgres=# select * from pg_stat_replication;
-[ RECORD 1 ]-----+-----
pid                | 1811
usesysid           | 16388
username           | replicarole
application_name    | pg2
client_addr        | 192.168.122.11
client_hostname     |
client_port        | 43890
backend_start       | 2023-05-22 13:41:13.846757+00
backend_xmin        |
state               | streaming
sent_lsn            | 0/30001F8
write_lsn           | 0/30001F8
flush_lsn           | 0/30001F8
replay_lsn          | 0/30001F8
write_lag           |
flush_lag           |
replay_lag          |
sync_priority       | 1
sync_state          | sync
reply_time          | 2023-05-22 13:41:33.879308+00
```

As shown here, the primary server and standby servers are replicated in a synchronous way by `sync_state=sync`.

Cascading replication

We have explored how to create an asynchronous replica starting from a primary server. However, in some cases, we may need multiple asynchronous replicas, and the simplest way to do this is to hook a second replica machine to the primary machine with the procedure we have just seen. This procedure, however, could increase the load on the primary machine, so PostgreSQL offers an alternative to this: cascading physical replication. The schema we want to achieve is this:



Figure 17.4: Cascading Replication

In order to make our example work, we will use a third machine called pg3.

The machines will have the following IPs:

- PRIMARY (pg1): IP 192.168.122.10
- First replica (pg2): IP 192.168.122.11
- Second replica (pg3): IP 192.168.122.12

Now take the following steps:

1. Similarly to what we did before, let's configure the pg2 machine so that it can receive requests from the pg3 machine. We have to add this line to the `pg_hba.conf` file:

```
IPv4 local connections:
host replication replicarole 192.168.122.12/32 scram-sha-256
```

2. Now, we have to reload the PostgreSQL service:

```
root@pg2:# systemctl reload postgresql
```

3. On the pg2 machine, let's execute the following SQL command:

```
root@pg2:/usr/local/pgsql# su - postgres
postgres@pg2:~$ psql
psql (16)
```

```
Type "help" for help.
```

```
postgres=# SELECT * FROM pg_create_physical_replication_
slot('standby1');
 slot_name | lsn
-----+-----
 standby1 |
(1 row)
```

4. As before, we have created a reference slot for cascade replication. Now let's go to the pg3 machine and turn off the PostgreSQL service:

```
root@pg3:# systemctl stop postgresql
```

5. Let's delete the contents of the /var/lib/postgresql/16/main directory:

```
root@pg3:# rm -rf /var/lib/postgresql/16/main/*
```

6. As a PostgreSQL user, let's perform the basebackup procedure:

```
posroot@pg3:~# su - postgres
root@pg3$: pg_basebackup -h 192.168.122.11 -U replicarole -p 5432 -D
/var/lib/PostgreSQL/16/main -Fp -Xs -P -R -S standby1
Password:
32743/32743 kB (100%), 1/1 tablespace
```

7. At this point, we can restart the PostgreSQL service. As the root user, let's execute the following:

```
# root@pg3:~# systemctl start postgresql
```

At this point, we are done! If we query the `pg_stat_replication` view on the standby1 server, we will see that a second replica exists. Now our system has two replicas, and we have achieved the goal that we set ourselves.

8. This is `pg_stat_replication` on the primary server (pg1):

```
postgres=# select * from pg_stat_replication ;
-[ RECORD 1 ]-----+-----
 pid                | 14339
 usesysid           | 16390
 username           | replicarole
 application_name    | walreceiver
```

```

client_addr      | 192.168.122.11
client_hostname  |
client_port      | 38844
backend_start    | 2023-05-29 09:31:18.443699+00
backend_xmin     |
state           | streaming
sent_lsn         | 0/43F1E40
write_lsn        | 0/43F1E40
flush_lsn        | 0/43F1E40
replay_lsn       | 0/43F1E40
write_lag        |
flush_lag        |
replay_lag       |
sync_priority    | 0
sync_state       | async
reply_time       | 2023-05-29 09:39:57.357726+00

```

9. This is `pg_stat_replication` on the standby1 server (pg2):

```

postgres=# select * from pg_stat_replication;
-[ RECORD 1 ]-----+-----
pid           | 13015
usesysid      | 16390
username      | replicarole
application_name | walreceiver
client_addr    | 192.168.122.12
client_hostname |
client_port    | 50990
backend_start  | 2023-05-29 09:39:26.994899+00
backend_xmin   |
state         | streaming
sent_lsn      | 0/43F1E40
write_lsn     | 0/43F1E40
flush_lsn     | 0/43F1E40
replay_lsn    | 0/43F1E40
write_lag     |
flush_lag     |
replay_lag    |

```



```

sync_priority | 0
sync_state    | async
reply_time    | 2023-05-29 09:39:47.031691+00

```

Thus, we have learned how cascading replication works. Cascading replication can be useful when we want to decrease the load on the primary machine, for example, as the basis of a **high availability (HA)** system based on three PostgreSQL servers, without loading the primary server.

Delayed replication

In some cases, it could be useful to have a delayed replica; in PostgreSQL, to achieve this goal, we can use the `recovery_min_apply_delay` on the settings of the replica server.

For example, on the replica server, if we put this setting at the end of the `postgresql.conf`:

```
recovery_min_apply_delay = 5000
```

and we make a reload of the `postgresql` service on the replica server:

```
root@pg2:~# systemctl reload postgresql
```

we can see that the replica is 5 seconds behind the primary because the time unit used on `recovery_min_apply_delay` is milliseconds.

Using a delay on the replica server means that WAL files are regularly downloaded from the primary server, but they are processed with the delay specified on the parameter `recovery_min_apply_delay`.

Promoting a replica server to a primary

If a primary goes down, on the log of the replica server, we receive this kind of error:

```

LOG:  waiting for WAL to become available at 0/4000078
2023-05-23 07:22:14.813 UTC [1137] FATAL:  could not connect to the
primary server: connection to server at "192.168.122.10", port 5432
failed: No route to host
Is the server running on that host and accepting TCP/IP connections?

```

This means that the replica server no longer receives WAL files from the primary; if this scenario happens, it is possible to promote the replica node to the primary; to achieve this goal, on the replica node, as a `postgres` user, we have to execute this statement:

```
postgres@pg2:~$ pg_ctl promote -D /var/lib/PostgreSQL/16/main
```

```
waiting for server to promote..... done
server promoted
```

After executing this statement on the log file, we will see something like:

```
2023-05-23 07:25:33.719 UTC [1078] LOG:  received promote request
2023-05-23 07:25:34.909 UTC [1211] FATAL:  could not connect to the
primary server: connection to server at "192.168.122.10", port 5432
failed: No route to host

                Is the server running on that host and accepting TCP/IP
connections?
2023-05-23 07:25:34.910 UTC [1078] LOG:  waiting for WAL to become
available at 0/4000078
2023-05-23 07:25:34.910 UTC [1078] LOG:  redo done at 0/4000028 system
usage: CPU: user: 0.00 s, system: 0.00 s, elapsed: 246.25 s
2023-05-23 07:25:34.928 UTC [1078] LOG:  selected new timeline ID: 2
2023-05-23 07:25:35.003 UTC [1078] LOG:  archive recovery complete
2023-05-23 07:25:35.014 UTC [1076] LOG:  checkpoint starting: force
2023-05-23 07:25:35.015 UTC [1075] LOG:  database system is ready to
accept connections
```

This means that the replica server has been promoted to the primary, and now it is possible to make write operations on this node. In this section, we have learned how to promote a replica server to a primary, but there are a couple of things we have to keep in mind:

- PostgreSQL doesn't complete this procedure automatically (without third-party tools).
- After promoting the standby replica to the primary, the old primary becomes unrecoverable.
- If the primary goes down, we can suffer data loss if not all the WAL files have been synchronized prior to the server going down.

Summary

In this chapter, we introduced the concept of physical replication. We started by reviewing and deepening our knowledge of WAL segments from previous chapters. We have introduced, seen, and configured an asynchronous physical replica and a synchronous physical replica. We looked at the difference between the two modes, and we saw how easy it is to switch from one mode to another. We then explored some useful tools to monitor replicas and check their good health. In the next chapter, we will use the concepts that we have discussed in this chapter to address the topic of logical replication.